

# GGD: Gravitational Gradient Descent

Dhruv Baweja

*An interim summary of work in progress*

---

## The idea

Momentum-based optimizers get described in the language of physics constantly — a ball rolling downhill, gaining momentum, slowed by friction. Almost none of that is literal. It adds velocity and inertia to gradient descent, but never computes an actual force field, mass, or potential energy.

GGD started from asking what happens if you don't stop at the metaphor. It treats the model's parameters as the position of a point mass on the loss surface, keeps a running estimate of local mass from recent gradient statistics, and defines an actual potential energy that grows as the parameters drift away from a reference point that gets recentered periodically. The total energy — kinetic plus potential gates a drag term: high energy means the optimizer moves more freely, low energy means it settles, a bit like an orbiting body only spiraling inward once it's lost enough energy to escape.

## How it got here

Earlier versions leaned much harder on the physics, mass and pull direction both came from the actual local curvature of the loss surface, via Hessian-vector products and Lanczos iteration, with a damped-Newton step deciding the direction of pull. Across roughly forty controlled experiments, I found that most attempts to make the physics *more* literal — richer curvature estimates, geometry-aware corrections to the pull direction, more elaborate energy bookkeeping either did nothing or made things worse, usually by amplifying noise already present in the curvature estimate. A handful of ordinary fixes helped consistently: correcting a startup bias in a moving average, scaling a velocity limit to model size, making sure gradient and curvature were always computed on the same batch.

The biggest single change came from simplifying rather than adding: replacing the Hessian-based curvature estimate entirely with a much cheaper first-order estimate of local mass, in the style of Adam's gradient second-moment tracking, matched or beat every more elaborate curvature-aware version I'd built at a fraction of the compute. The version below is that simplified, first-order form.

The energy term is doing a specific job here: when it's high, the optimizer treats the current basin as unconfirmed and keeps some freedom to move sideways out of it; as energy drops, that freedom gets damped down, and the optimizer commits more fully to descending. Several of the failed attempts above tried to make that energy estimate richer by feeding it more curvature information, the surprising part was that this

consistently made the signal noisier rather than more informative, which is part of why the simpler, first-order version ended up ahead.

## Results

I benchmarked GGD against AdamW, Muon, Shampoo, and SOAP, with a dedicated hyperparameter search and multiple random seeds for every optimizer, including the baselines.

**Tabular classification** (Forest Cover Type, Digits, Wine, Breast Cancer; mean final accuracy):

| Optimizer  | Accuracy     | Wall time (s) |
|------------|--------------|---------------|
| SOAP       | 92.4%        | 0.90          |
| AdamW      | 92.2%        | 0.52          |
| <b>GGD</b> | <b>91.9%</b> | <b>0.52</b>   |
| Muon       | 91.8%        | 1.41          |
| Shampoo    | 89.0%        | 3.46          |

GGD lands within a point of AdamW and SOAP, ahead of Muon and Shampoo, at the same wall-clock cost as AdamW and with the most consistent per-step timing of anything I tested.

**Image classification** (CIFAR-<sup>10</sup>/<sub>100</sub>, small CNN and ResNet-18; mean final accuracy):

| Optimizer  | Accuracy     | Wall time (s) |
|------------|--------------|---------------|
| SOAP       | 31.8%        | 24.1          |
| AdamW      | 28.3%        | 19.0          |
| Muon       | 27.8%        | 19.3          |
| <b>GGD</b> | <b>21.9%</b> | <b>19.9</b>   |
| Shampoo    | 15.2%        | 121.3         |

GGD stays ahead of Shampoo and runs at close to AdamW/Muon's cost, but trails all three by a real margin.

## The open problem

The image-classification gap is the thing I don't have a confirmed explanation for yet. My first guess — unreliable curvature estimates at high parameter counts turned out to be incomplete: the current, curvature-free version of GGD still shows the same qualitative gap, just narrower and much cheaper to run. I'm still working out what's actually driving it.

## Where this stands

This is a progress report, I expect both the method and these numbers to keep changing.